
RAPPORT DE PROJET DEVOPS

Développement d'un Système d'Évaluation Automatisé au Dakar Institute of Technology



Description:

Le projet consiste à créer un logiciel interne d'évaluation des enseignements pour le Dakar Institute of Technology (DIT). Actuellement, le Dakar Institute of Technology (DIT) utilise Typeform pour les rapports d'évaluation des enseignements. Bien que Typeform soit un outil pratique, il présente plusieurs limitations qui affectent l'efficacité et la pertinence des évaluations. Les principales limitations incluent un contrôle limité sur les données, des options de personnalisation restreintes et des capacités d'analyse de données insuffisantes. Face à ces défis, le DIT a décidé de développer un logiciel interne dédié à l'évaluation des enseignements pour surmonter ces problèmes et améliorer la qualité de ses processus.

Réalisé par le groupe 4:

Mame Bousso SECK

Abdoulaye NDOUR

Mouhamed DIOP

Afdel Desmond KOMBOU

Sous la supervision:

Mr Madické DIOP

Plan Détaillé

I. Contexte et Objectifs du Projet

1. Contexte du Projet

- Présentation du Dakar Institute of Technology (DIT)
- Problématiques actuelles avec l'utilisation de Typeform pour les évaluations
- Limitations de Typeform : contrôle des données, personnalisation, analyse des données

2. Objectifs du Projet

- Développement d'un logiciel interne pour l'évaluation des enseignements
- Amélioration de la sécurité et de la confidentialité des données
- Personnalisation des formulaires d'évaluation
- Intégration d'outils d'analyse de données avancés
- Automatisation de la génération de rapports détaillés

II. Technologies et Outils Utilisés

III. Organisation et Planification du Projet

1. Gestion des Tâches avec Trello

- Création de tableaux (À faire, En cours, Terminé)
- Assignment des tâches aux membres de l'équipe

2. Gestion du Code Source avec GitHub

- Création du repository GitHub
 - Initialisation du projet local avec Git
 - Push du code source vers GitHub
-

IV. Développement de l'Application

1. Configuration de l'Environnement de Développement

- Installation des dépendances avec pip
- Configuration de l'IDE (VS Code)

2. Développement du Backend

- Modélisation de la base de données
- Création des tables (Classes, Matières, Réponses)
- Sécurisation des données (hachage des mots de passe)

3. Développement du Frontend

- Conception de l'interface utilisateur
- Création des formulaires d'évaluation pour les étudiants
- Interface d'administration pour la gestion des classes, matières et réponses

V. Test et Déploiement

1. Configuration du Pipeline Jenkins

- Intégration de Jenkins avec GitHub
- Automatisation des tests et du déploiement

2. Conteneurisation avec Docker

- Création du Dockerfile
- Configuration de docker-compose.yml
- Déploiement local avec Docker

3. Déploiement avec Kubernetes

- Création du fichier deployment.yaml
 - Déploiement de l'application et de la base de données sur un cluster Kubernetes
-

- Configuration de trois réplicas pour l'application web

VI. Conclusion et Perspectives

1. Résumé du Projet

- Réalisation des objectifs fixés
- Utilisation des outils modernes de développement et de déploiement

2. Perspectives

- Application des connaissances acquises dans de futurs projets
 - Améliorations potentielles de l'application
 - Interface utilisateur
-

I. Contexte et Problématique

Le Dakar Institute of Technology (DIT) s'engage à offrir une éducation de haute qualité, adaptée aux besoins changeants de ses étudiants et de la société. L'évaluation continue des enseignements est cruciale pour maintenir et améliorer cette qualité. Actuellement, le DIT utilise Typeform pour recueillir et analyser les évaluations des enseignements. Cependant, cette approche présente plusieurs défis qui limitent l'efficacité et la pertinence des évaluations. D'une part, l'utilisation de Typeform implique que les données collectées sont hébergées sur des serveurs externes, ce qui restreint le contrôle direct de l'institution sur ces informations sensibles. Cela pose des problèmes de confidentialité et de sécurité des données, d'autant plus critiques dans un contexte éducatif. En outre, les fonctionnalités offertes par Typeform en termes de personnalisation des formulaires et d'analyse des données sont limitées. Cela empêche le DIT d'adapter précisément les évaluations aux spécificités des différents cours et départements et de réaliser des analyses approfondies nécessaires pour des améliorations pédagogiques efficaces.

Face à ces limitations, le DIT se trouve dans l'obligation de repenser son approche de l'évaluation des enseignements vers une solution plus contrôlée et personnalisable. La mise en place d'un logiciel interne dédié à cette tâche apparaît comme une solution stratégique pour surmonter ces défis. Un tel logiciel permettrait non seulement de garantir la sécurité et la confidentialité des données, mais aussi de bénéficier d'une personnalisation complète des formulaires d'évaluation, d'intégrer des outils d'analyse de données avancés et d'automatiser la génération de rapports détaillés. Ainsi, le DIT pourrait améliorer la qualité de ses enseignements en utilisant des données précises pour répondre aux besoins des étudiants et des enseignants, tout en rendant les évaluations plus efficaces et en réduisant les coûts à long terme.

I. Technologies Utilisées



Trello est un outil de gestion de projet visuel qui permet de suivre les tâches et d'organiser le travail en utilisant des cartes et des tableaux. Il est utilisé pour planifier, assigner des tâches et suivre la progression du projet de gestion des étudiants.

Slack est une plateforme de collaboration et de communication en ligne qui permet aux équipes de travail de communiquer et de partager des informations de manière plus efficace.



VS Code est un éditeur de code source puissant et léger développé par Microsoft. Il est utilisé pour écrire, déboguer et tester le code Flask, offrant une multitude d'extensions pour améliorer le développement et la productivité.

Flask est un micro-framework de développement web en Python, léger et flexible, qui permet de créer rapidement des applications web et mobiles personnalisées. Il est basé sur Werkzeug pour la partie serveur/WSGI et utilise Jinja 2 pour le templating.



GitHub est une plateforme de gestion de code source utilisant Git. Il est utilisé pour héberger le code source du projet, permettre la collaboration entre les membres de l'équipe et gérer les versions du code.

Jenkins est un outil d'intégration continue et de livraison continue (CI/CD) open-source. Il est utilisé pour automatiser les tests et les déploiements du projet, assurant que le code est constamment testé et que les nouvelles versions sont déployées de manière fiable.



Docker est une plateforme de conteneurisation qui permet de créer, déployer et exécuter des applications dans des conteneurs isolés. Il est utilisé pour emballer l'application Flask avec toutes ses dépendances, assurant une exécution cohérente dans différents environnements.

Kubernetes, souvent abrégé en K8s (car il y a 8 lettres entre le K et le s), est une plateforme open source pour l'orchestration de conteneurs. Elle permet d'automatiser le déploiement, la mise à l'échelle et la gestion des applications conteneurisées.



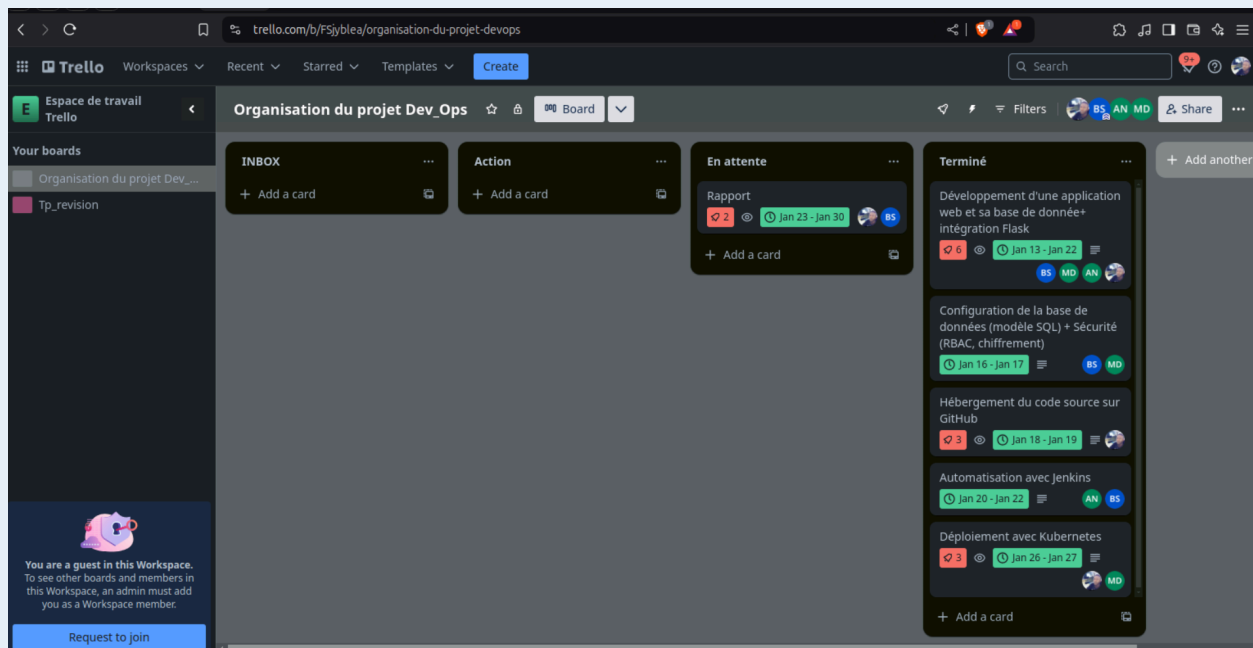
II. Organisation et Planification du Projet

Note:

Cette section décrit comment le projet a été organisé et planifié en utilisant des outils collaboratifs comme **Trello** et **Slack**. L'équipe a utilisé Trello pour gérer les tâches, en les classant en différentes listes (**À faire**, **En cours**, **Terminé**) et en assignant des responsabilités spécifiques à chaque membre. Slack a été intégré pour recevoir des alertes et maintenir une communication efficace. Le repository **GitHub** a été créé pour la gestion du code source, permettant une collaboration fluide et une gestion de version efficace.

1. Utilisation de Trello pour la Gestion des Tâches

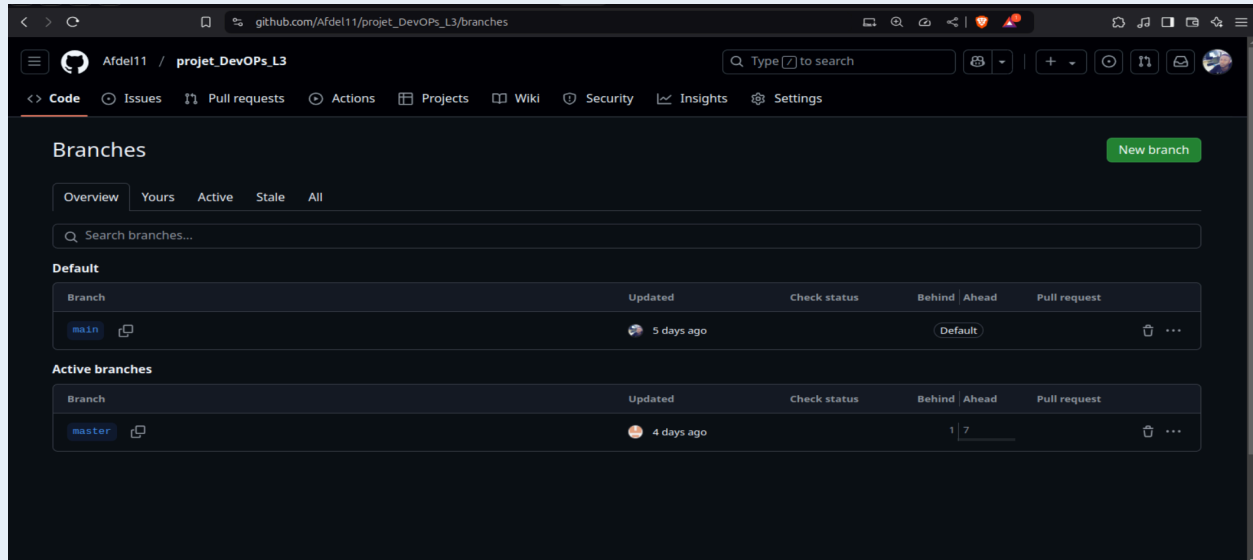
À faire, En cours, Terminé



2. Création et Gestion du Repository GitHub

Cette section explique comment nous avons configuré notre projet Flask pour utiliser **Git**, en créant un **repository local** et en le connectant à un repository distant sur **GitHub**.

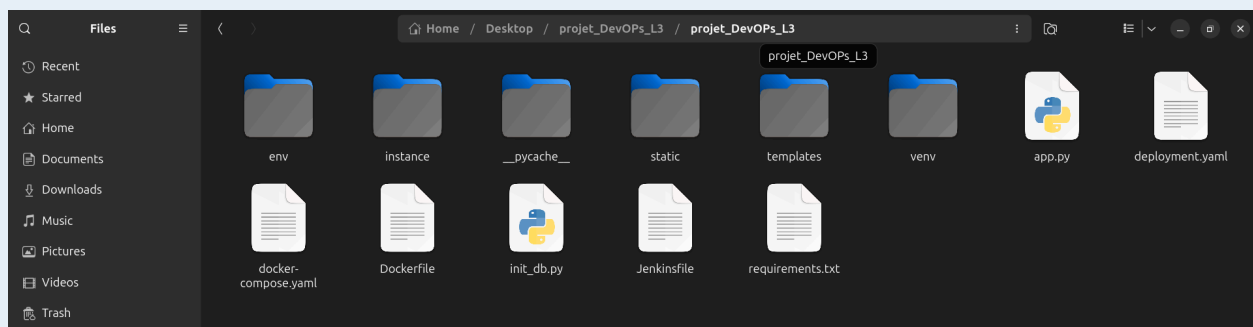
- Création d'un repository sur GitHub avec le même nom que le projet local



- Initialisation du projet local en tant que repository Git

Commande d'initialisation du projet

- *git init*
- *git add .*
- *git commit -m "Initial commit"*

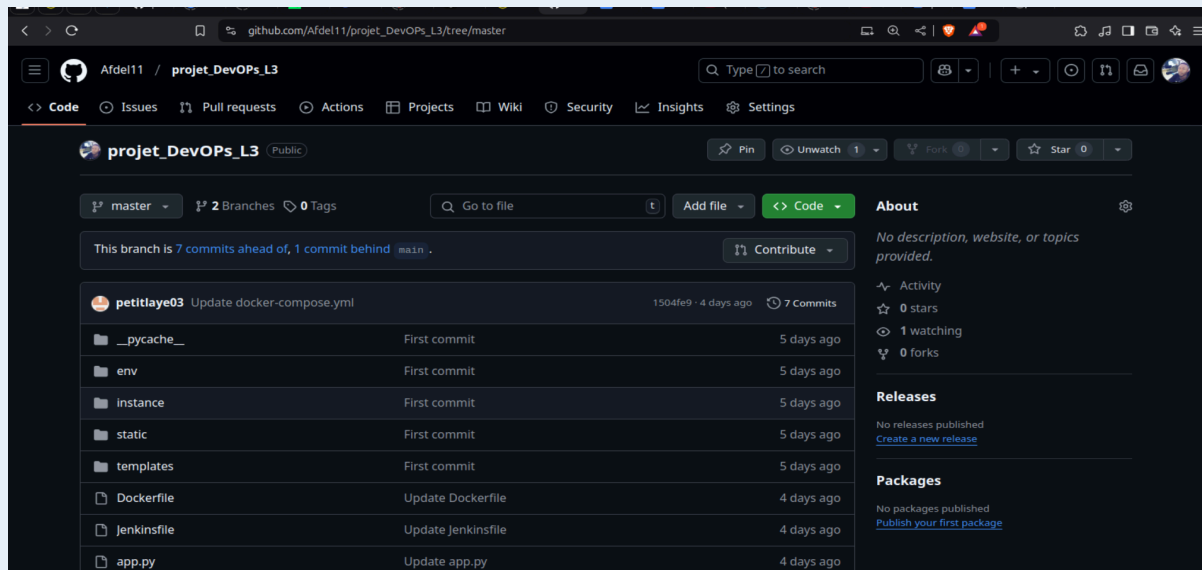


- Envoi du code source vers le dépôt distant GitHub

- Créez un nouveau **repository** sur GitHub depuis l'interface web. Copiez l'URL de ce nouveau repository.
- *git remote add origin https://github.com/Afdel11/projet_DevOPs_L3.git*

→ *git push -u origin master*

Avec ces commandes, notre projet Flask est maintenant configuré pour une gestion de version efficace via **Git** et **GitHub**, permettant une collaboration harmonieuse et un suivi des modifications apportées au **code source**.



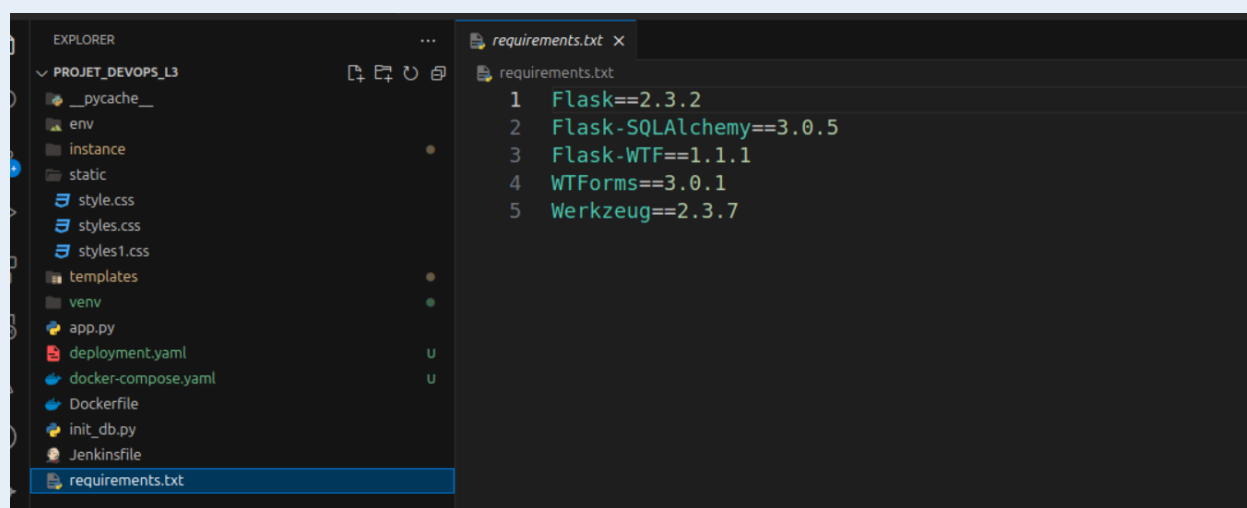
La préparation du projet étant terminée, nous pouvons maintenant passer au développement de l'application proprement dite.

III. Développement de l'Application

Note:

Dans cette section, nous détaillons le processus de développement de l'application **Flask** pour la gestion des évaluations, en utilisant **Visual Studio Code**. Les fonctionnalités ont été implémentées et testées dans l'environnement de développement.

1. Installation des dépendances et configuration de l'IDE



Flask : Microframework web léger et flexible pour Python, idéal pour créer des applications web simples et complexes.

Flask-SQLAlchemy : Extension de Flask qui intègre SQLAlchemy pour simplifier l'interaction avec les bases de données relationnelles.

Flask-WTF : Extension de Flask qui facilite l'intégration de WTForms pour la gestion et la validation des formulaires web.

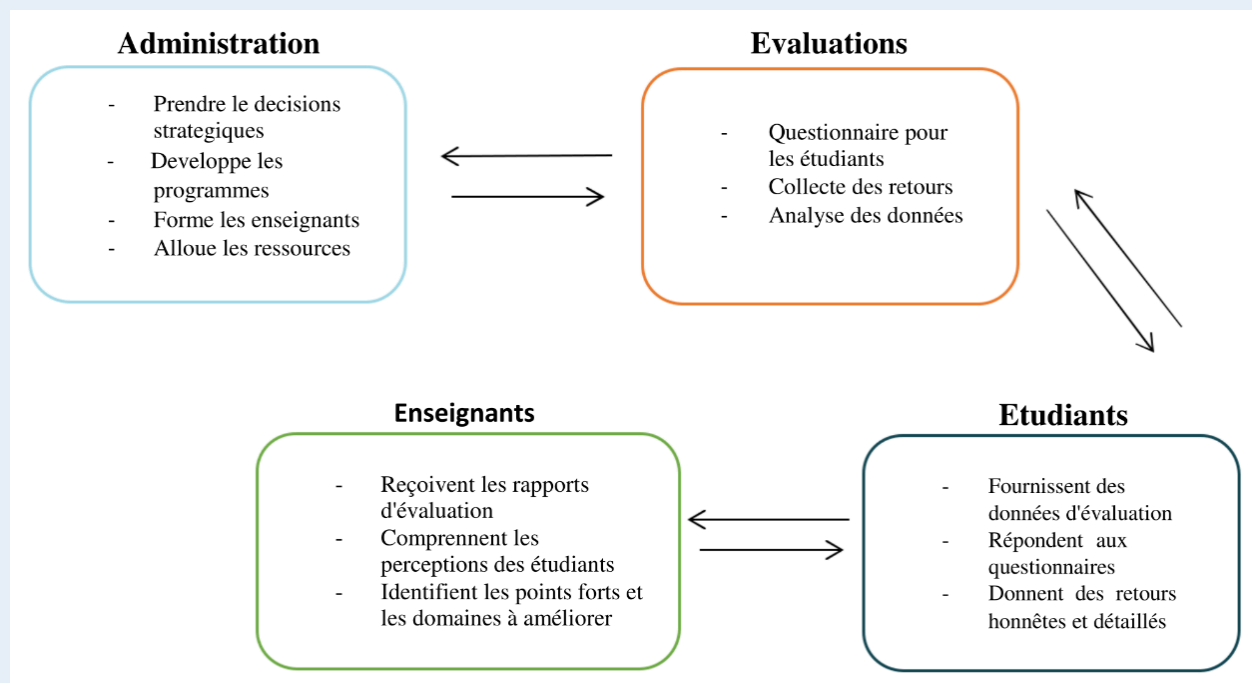
WTForms : Bibliothèque pour la création et la validation de formulaires web en Python, indépendante de Flask.

Werkzeug : Bibliothèque utilitaire WSGI, utilisée par Flask pour gérer les requêtes et réponses HTTP.

1. Développement du Backend projet

Modèle de données

Le modèle de données du logiciel d'évaluation des enseignements du Dakar Institute of Technology (DIT) est structuré pour répondre aux besoins des évaluations des enseignements et inclura des tables principales telles que 'Classes', 'Matières', 'Réponses'. Chaque formulaire pourra contenir plusieurs questions, et chaque réponse sera liée à une question spécifique. Le modèle doit permettre des relations flexibles entre les formulaires, les questions et les réponses pour faciliter l'analyse et la génération de rapports. Des mécanismes de validation des données seront intégrés pour garantir l'intégrité des informations et leur conformité aux exigences de sécurité et de confidentialité.



Voici les principaux composants du modèle de données :

a. Tables Principales :

Classe : Contient les informations sur les classes disponibles.

Id : est la clé primaire (PK)

Nom : stocke le nom de la classe.

Matière: Contient les informations sur les matières disponibles.

Id : est la clé primaire (PK).

Nom : stocke le nom de la matière.

Réponse: Stocke les réponses aux sondages.

Id : est la clé primaire (PK)

'nom_classe' et 'nom_matière' : sont des clés étrangères (FK) qui font référence aux tables Classe et Matière.

Les autres colonnes stockent les réponses aux questions du sondage, telles que l'implication, la satisfaction, etc.

classe		matiere		Reponse	
id	integer	id	integer	id	integer
nom	varchar	nom	varchar	nom_classe	fk
				nom_matiere	fk
				implication	int
				connaissances_initiales	int
				connaissances_actuelles	int
				motivation_professeur	int
				outils_methodologie	int
				exemples_exercices	int
				clarté_explications	int
				compétences_pratiques	varchar
				organisation_cours	varchar
				satisfaction_générale	int
				commentaires	varchar

b. L'Application flask

Configuration de l'application :

L'application utilise Flask pour créer des routes et gérer les requêtes HTTP.

Une base de données SQLite est configurée via SQLAlchemy pour stocker les données (classes, matières, réponses aux sondages).

Des formulaires sont créés avec Flask-WTF et WTForms pour collecter les réponses des utilisateurs.

Fonctionnalités principales :

Sondage : Les utilisateurs peuvent remplir un formulaire de sondage pour évaluer un cours. Les données sont stockées dans la base de données.

Administration : Un administrateur peut se connecter pour gérer les classes, les matières, et consulter les résultats des sondages.

Rapports : L'administrateur peut générer des rapports basés sur les réponses des utilisateurs, avec des moyennes et des statistiques.

Gestion des comptes : L'administrateur peut changer ses identifiants de connexion.

Sécurité :

Les mots de passe sont hachés avec Werkzeug pour une sécurité accrue.

L'accès à certaines pages est restreint aux utilisateurs connectés.

Gestion des données :

Les classes et matières peuvent être ajoutées ou supprimées via l'interface d'administration.

Les réponses aux sondages peuvent être réinitialisées.

```

1  from flask import Flask, render_template, request, redirect, url_for, session, flash
2  from flask_sqlalchemy import SQLAlchemy
3  from flask_wtf import FlaskForm
4  from wtforms import RadioField, SelectField, TextAreaField, SubmitField
5  from wtforms.validators import DataRequired
6  from werkzeug.security import generate_password_hash, check_password_hash
7
8  app = Flask(__name__)
9  app.secret_key = 'your_secret_key' # Remplacez par une clé secrète appropriée
10
11 # Configuration de la base de données
12 app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
13 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
14 db = SQLAlchemy(app)
15
16 # Modeles de base de données
17 class Classe(db.Model):
18     id = db.Column(db.Integer, primary_key=True)
19     nom = db.Column(db.String(100), nullable=False)
20
21 class Matiere(db.Model):
22     id = db.Column(db.Integer, primary_key=True)
23     nom = db.Column(db.String(100), nullable=False)
24
25 class SurveyResponse(db.Model):
26     id = db.Column(db.Integer, primary_key=True)
27     class_name = db.Column(db.String(50), nullable=False)
28     subject_name = db.Column(db.String(50), nullable=False)
29     involvement = db.Column(db.Integer, nullable=False)
30     initial_knowledge = db.Column(db.Integer, nullable=False)
31     current_knowledge = db.Column(db.Integer, nullable=False)

```

Templates HTML :

L'application utilise des templates HTML pour afficher les pages (accueil, sondage, administration, etc.).

En résumé, cette application est un outil complet pour collecter et analyser les feedbacks sur des cours, avec une interface d'administration pour gérer les données et générer des rapports.

2. Configuration du Frontend du Projet

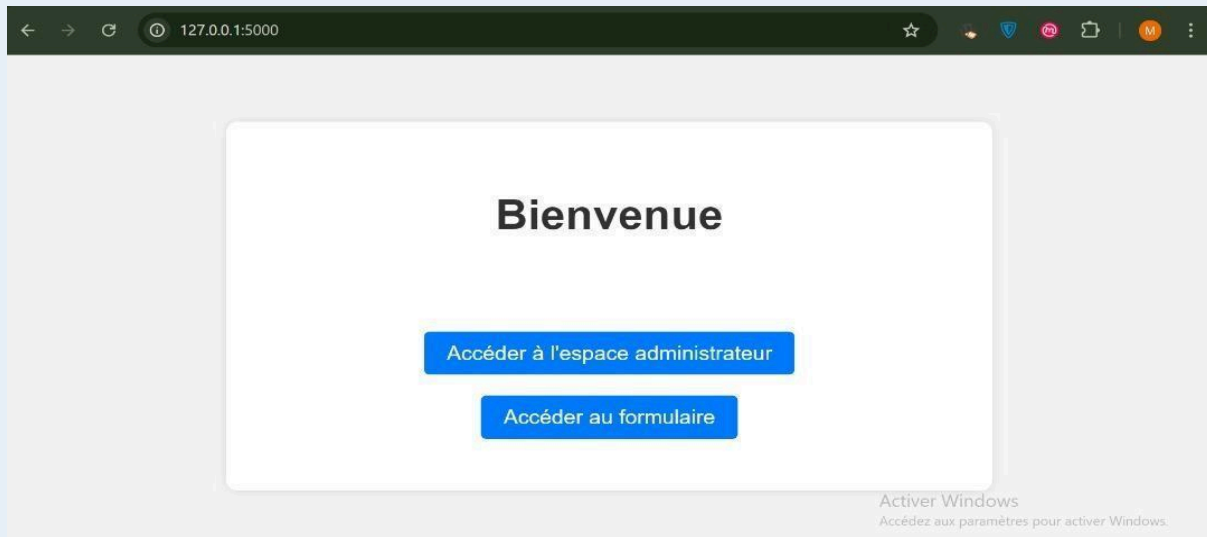
a. Interface Utilisateur

L'interface utilisateur (UI) du logiciel d'évaluation des enseignements du Dakar Institute of Technology (DIT) est conçue pour être intuitive, accessible et adaptée aux besoins de différents types d'utilisateurs : étudiants et administrateurs. Elle vise à offrir une expérience utilisateur fluide et agréable, en mettant l'accent sur la simplicité et l'efficacité.

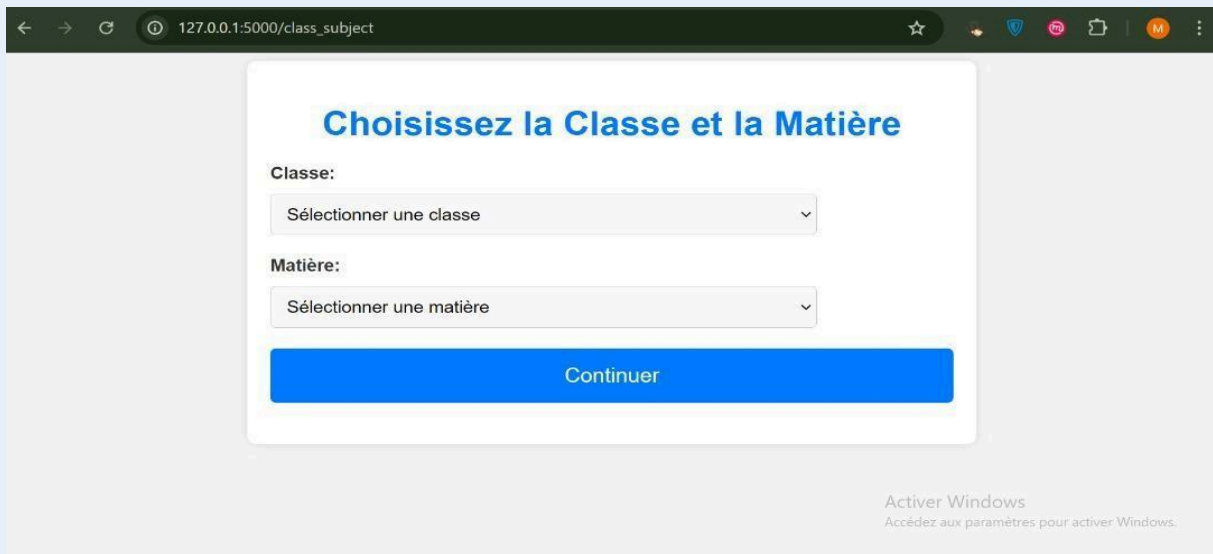
a. Interface pour les étudiants :

- **Accueil**

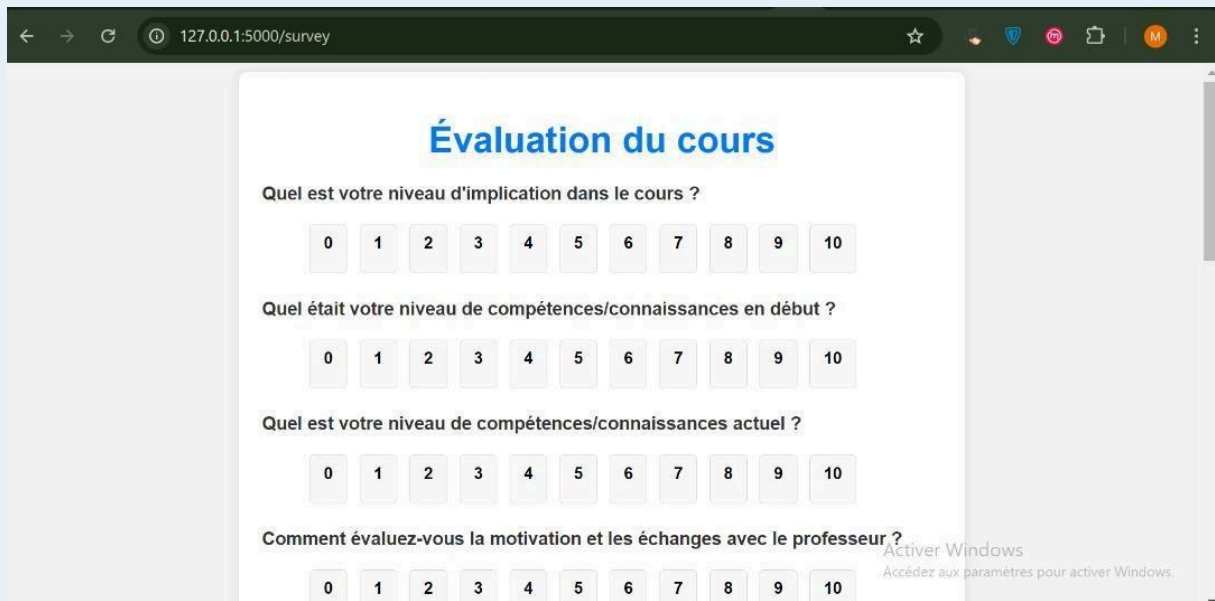
Ici les étudiants n'ont accès qu'au formulaire



- **Choix de classe et de matière :**



- **Formulaires :**

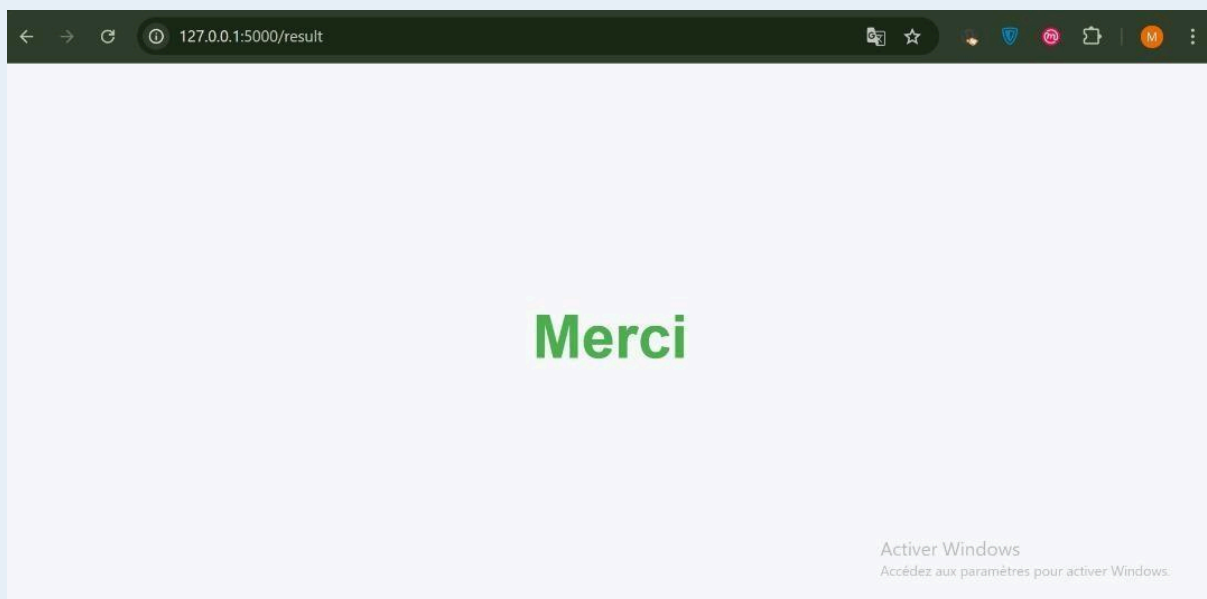


The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/survey". The main content area features a form titled "Évaluation du cours" in blue. The form contains four questions, each with a horizontal row of buttons numbered 0 to 10 for rating. The questions are:

- Quel est votre niveau d'implication dans le cours ?
- Quel était votre niveau de compétences/connaissances en début ?
- Quel est votre niveau de compétences/connaissances actuel ?
- Comment évaluez-vous la motivation et les échanges avec le professeur ?

At the bottom right of the browser window, there is a Windows watermark that says "Activer Windows" and "Accédez aux paramètres pour activer Windows."

L'étudiant doit remplir le formulaire et le soumettre. Une fois que toutes les informations seront remplies, il sera redirigé vers une page de confirmation.



b. Interface Administrateur

- **Page connexion**

Ici l'administrateur doit s'identifier pour pouvoir accéder à la partie administrative

- **Page d'accueil administrateur :**

- **Modification des identifiants :**

La modification des identifiants permet à l'administrateur de changer son nom d'utilisateur et son mot de passe après avoir vérifié ses anciennes informations d'identification.

Changer les Identifiants Administrateurs

Ancien Nom d'utilisateur:

Ancien Mot de Passe:

Nouveau Nom d'utilisateur:

Nouveau Mot de Passe:

[Retour à l'administration](#) [Mettre à jour](#)

- **Affichage des réponses :**

Ici on a récupéré la classe, la matière et le commentaire de l'étudiant avec une option de filtre pour différentes classes.

Après un sondage, l'administrateur a la possibilité de réinitialiser la table afin de ne pas avoir des confusions lors d'un second sondage.

Réponses au Sondage

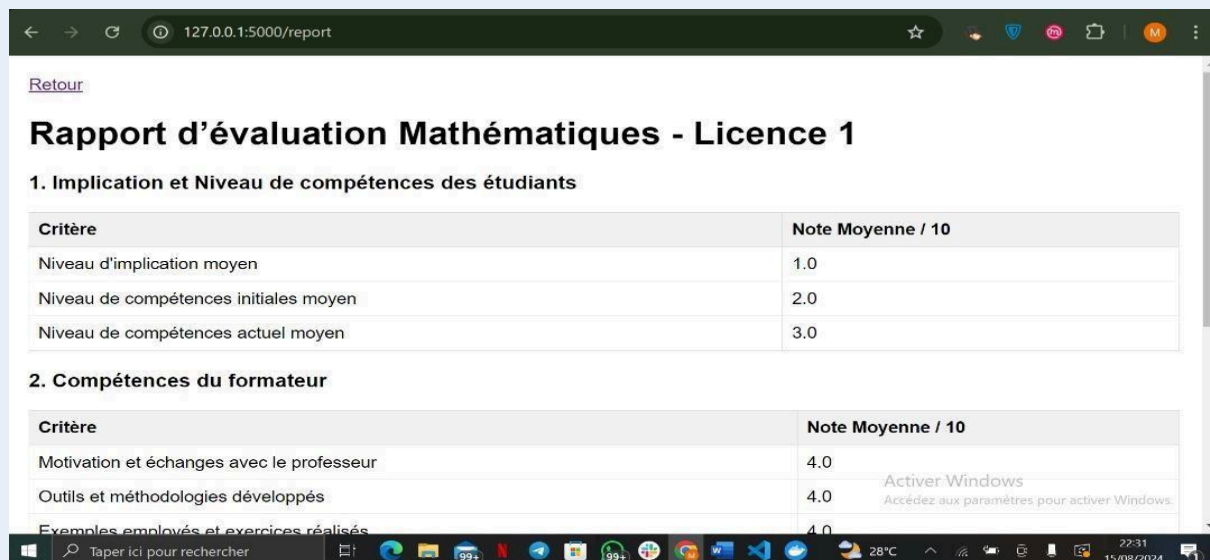
Filtrer par classe: Toutes les classes [Filtrer](#)

Classe	Matière	Commentaire
Licence 2	Mathématiques	xcvbn,kk
Licence 2	Mathématiques	xcvbn,kk
Licence 2	Mathématiques	xcvbn,kk
Licence 2	Mathématiques	xcvbn,kk
Licence 2	Mathématiques	ddeqc c
Licence 1	SQL	vfrea
Licence 1	Mathématiques	bien

[Réinitialiser les Résultats](#)

- **Rapports et Statistiques :**

Génération de rapports agrégés pour l'analyse institutionnelle.



[Retour](#)

Rapport d'évaluation Mathématiques - Licence 1

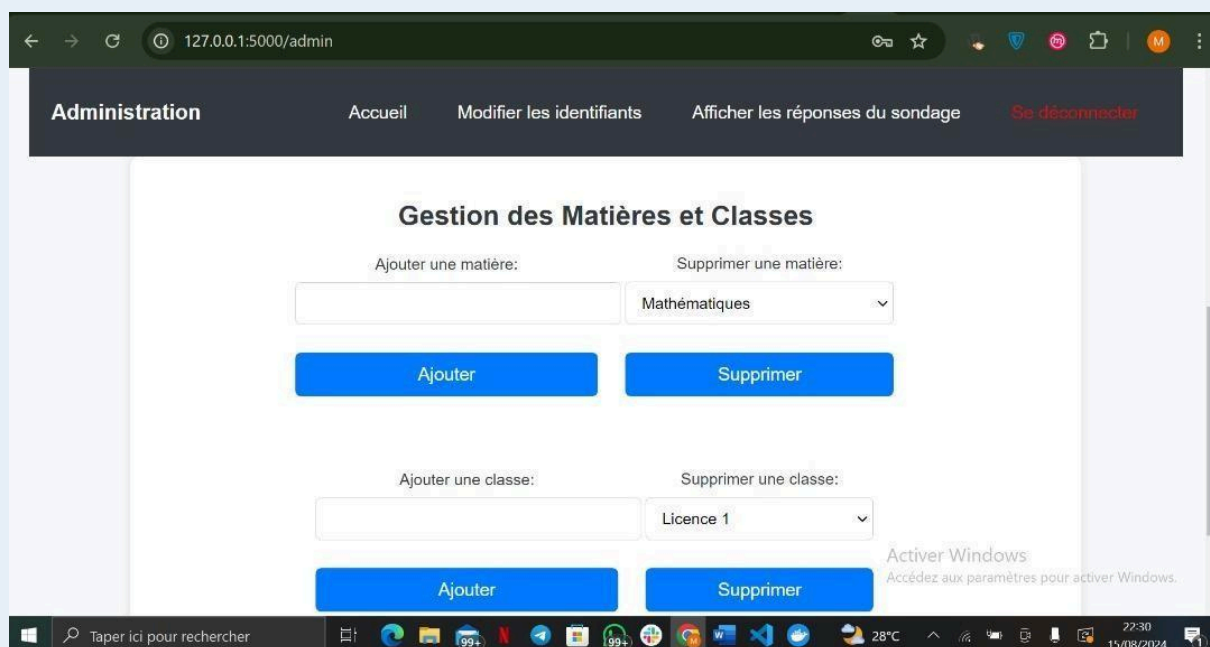
1. Implication et Niveau de compétences des étudiants

Critère	Note Moyenne / 10
Niveau d'implication moyen	1.0
Niveau de compétences initiales moyen	2.0
Niveau de compétences actuel moyen	3.0

2. Compétences du formateur

Critère	Note Moyenne / 10
Motivation et échanges avec le professeur	4.0
Outils et méthodologies développés	4.0
Exemples employés et exercices réalisés	4.0

- **Gestion matières et classes :**



Administration Accueil Modifier les identifiants Afficher les réponses du sondage [Se déconnecter](#)

Gestion des Matières et Classes

Ajouter une matière: Supprimer une matière:

Ajouter une classe: Supprimer une classe:

L'administrateur a la possibilité d'ajouter et de supprimer un module de même que pour les class

IV. Test et Conteneurisation avec Jenkins

1. Création du DockerFile

Notre fichier `Dockerfile` est utilisé pour créer une image Docker pour notre application Python, généralement une application Flask. Voici une explication simple de chaque étape :

1. Image de base :

- **`FROM python:3.11-slim`** : Utilise une image légère de Python 3.11 comme point de départ.

2. Répertoire de travail :

- **`WORKDIR /app`** : Définit `/app` comme répertoire de travail dans le conteneur.

3. Copie des fichiers:

- **`COPY . /app`** : Copie tous les fichiers du projet local dans le répertoire `/app` du conteneur.

4. Installation des dépendances :

- **`RUN pip install --no-cache-dir -r requirements.txt`** : Installe les dépendances Python listées dans `requirements.txt`.

5. Initialisation de la base de données :

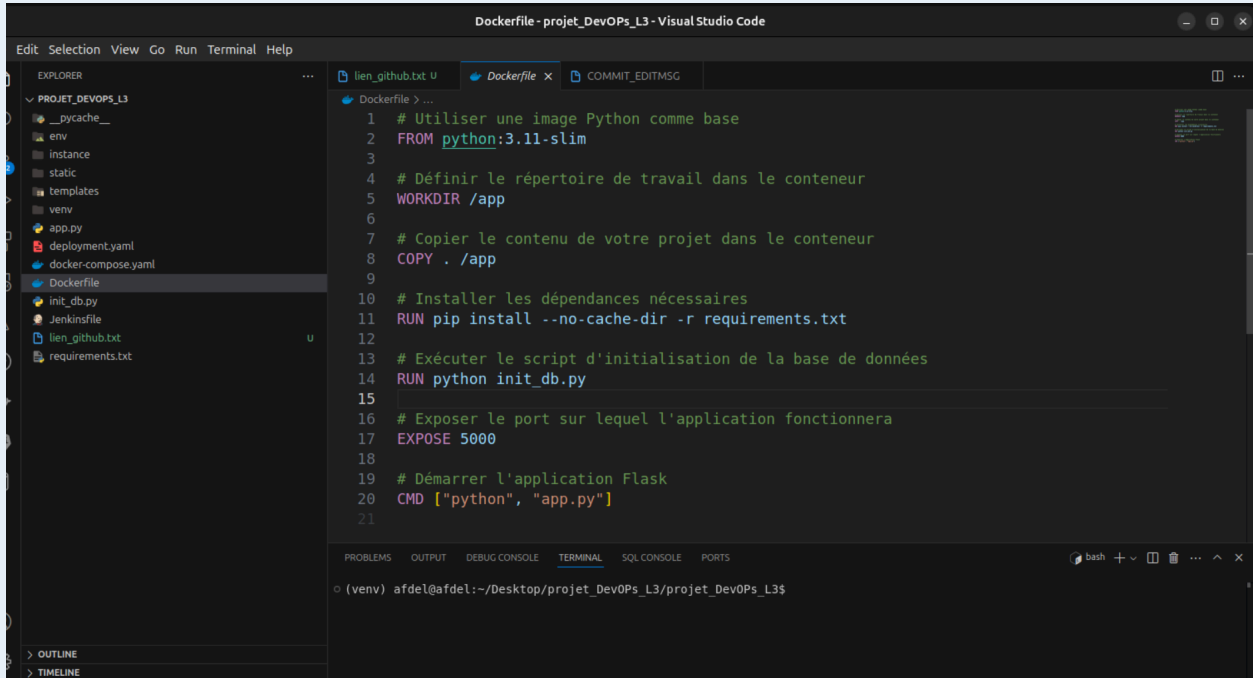
- **`RUN python init\_db.py`** : Exécute un script Python pour initialiser la base de données (si nécessaire).

6. Exposition du port :

- **`EXPOSE 5000`** : Ouvre le port 5000 pour permettre à l'application Flask de communiquer avec l'extérieur.

7. Démarrage de l'application :

- **CMD ["python", "app.py"]** : Démarre l'application Flask en exécutant `app.py`.



```

1 # Utiliser une image Python comme base
2 FROM python:3.11-slim
3
4 # Définir le répertoire de travail dans le conteneur
5 WORKDIR /app
6
7 # Copier le contenu de votre projet dans le conteneur
8 COPY . /app
9
10 # Installer les dépendances nécessaires
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Exécuter le script d'initialisation de la base de données
14 RUN python init_db.py
15
16 # Exposer le port sur lequel l'application fonctionnera
17 EXPOSE 5000
18
19 # Démarrer l'application Flask
20 CMD ["python", "app.py"]
21

```

Ce `Dockerfile` crée une image Docker pour une application Flask. Il installe les dépendances, initialise la base de données, expose le port 5000 et lance l'application. C'est une configuration typique pour conteneuriser une application Python.

2. Création du Docker Compose

Notre fichier `docker-compose.yml` définit deux services pour une application Flask : un service pour l'application web (`web`) et un autre pour la base de données (`db`). Voici une explication simple de chaque partie :

Services

1. Service `web` :

- **build: .** : Construit l'image Docker à partir du `Dockerfile` dans le répertoire courant.
- **ports: "5000:5000"** : Expose le port 5000 du conteneur sur le port 5000 de l'hôte.

- **`volumes`** :

- **`.`:/app`** : Monte le répertoire courant (code source) dans `/app` du conteneur pour le développement.

- **`db\_data:/app/instance`** : Monte un volume nommé `db\_data` pour stocker les données de l'application (comme la base de données SQLite).

- **`environment`** : Définit la variable d'environnement `FLASK\_ENV=development` pour activer le mode développement de Flask.

- **`depends\_on: db`** : Assure que le service `db` est démarré avant le service `web`.

2. Service `db` :

- **`image: busybox`** : Utilise une image légère `busybox` pour simuler un service de base de données.

- **`volumes: db\_data:/data`** : Monte le volume `db\_data` dans `/data` du conteneur pour stocker les données.

- **`command: sleep infinity`** : Garde le conteneur en vie indéfiniment (utile pour maintenir le volume actif).

Volumes

- **`db\_data`** :

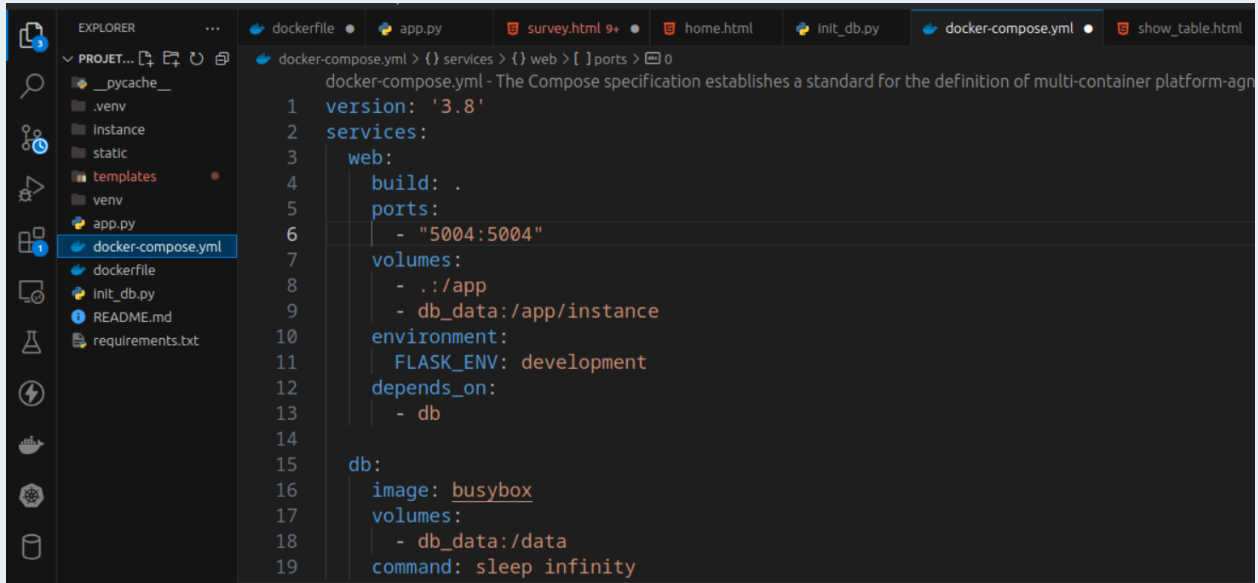
Définit un volume nommé `db\_data` pour stocker les données de la base de données de manière persistante.

Ce fichier `docker-compose.yml` configure deux services :

1. **web`** : Une application Flask qui s'exécute sur le port 5000, avec un montage du code source et un volume pour les données.

2. **`db`** : Un service minimaliste pour gérer un volume de données persistant.

C'est une configuration typique pour développer et tester une application Flask avec une base de données locale.



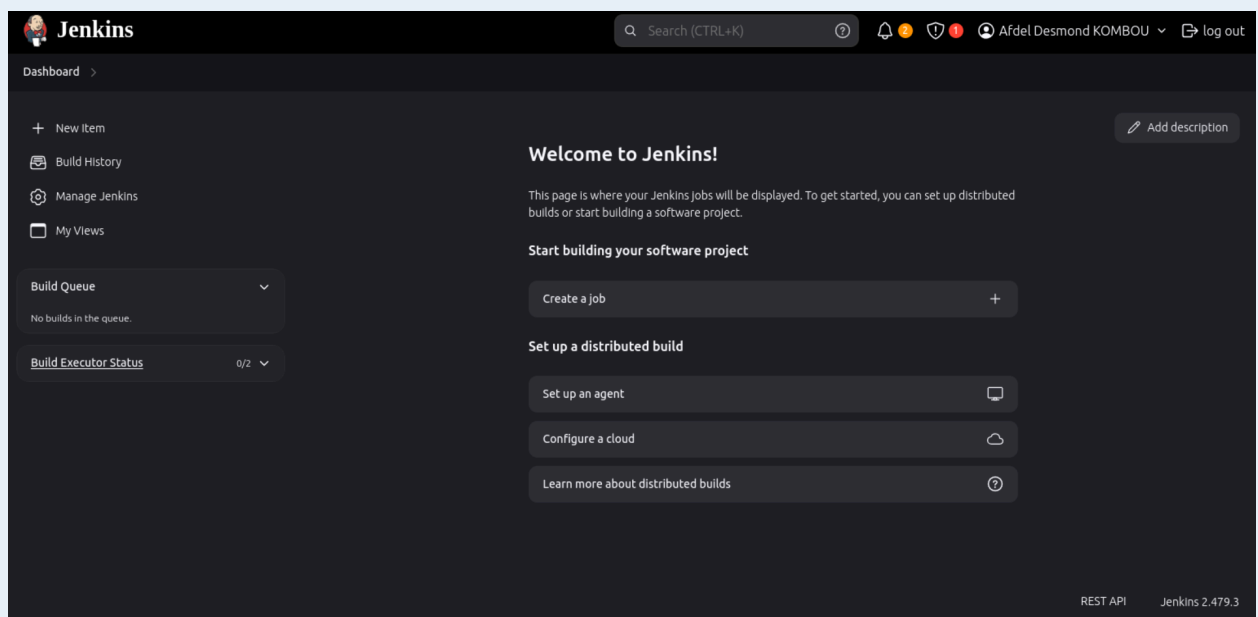
```

1 version: '3.8'
2 services:
3   web:
4     build: .
5     ports:
6       - "5004:5004"
7     volumes:
8       - ../app
9       - db_data:/app/instance
10    environment:
11      FLASK_ENV: development
12    depends_on:
13      - db
14
15  db:
16    image: busybox
17    volumes:
18      - db_data:/data
19    command: sleep infinity

```

3. Automatisation avec Jenkins

Installation et configuration Jenkins.



Notre pipeline Jenkins est conçu pour automatiser le processus de test et de déploiement d'une application en utilisant Docker. Voici une explication simplifiée :

Cloner le code :

Jenkins récupère le code source de l'application depuis un dépôt GitHub.

Construire les conteneurs :

Jenkins utilise Docker pour construire les images des conteneurs définies dans un fichier docker-compose.yml.

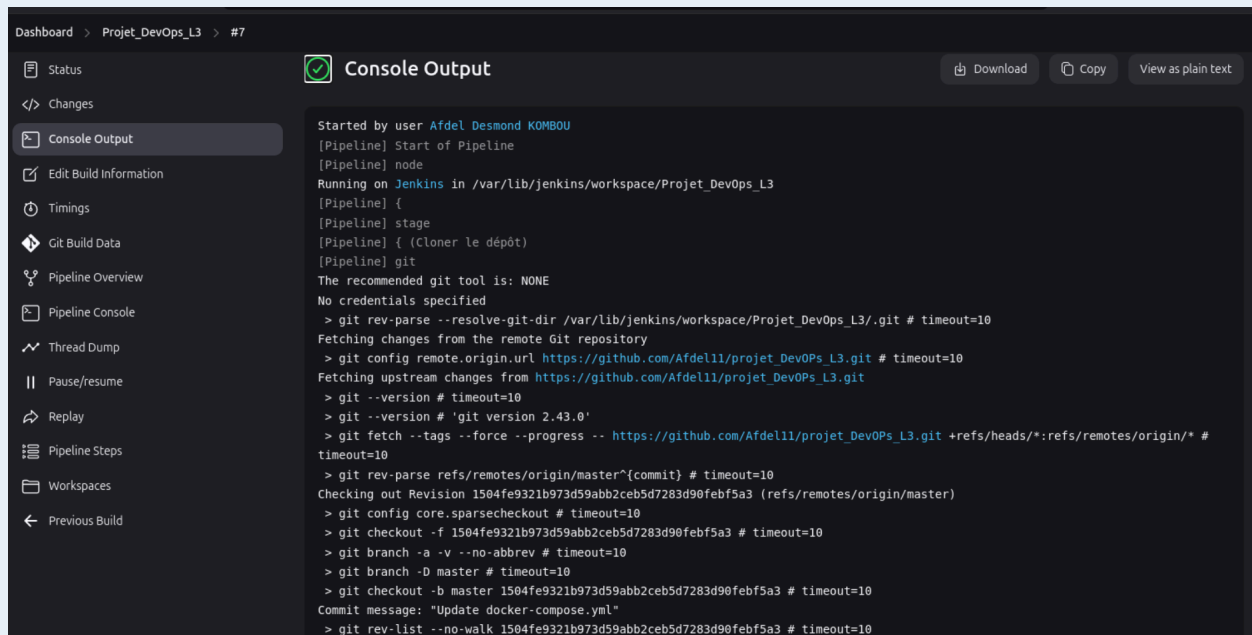
Déployer l'application :

Jenkins démarre les conteneurs en arrière-plan avec docker-compose up -d, ce qui permet de déployer l'application localement.

Résultat :

Si tout se passe bien, Jenkins affiche un message de succès.

En cas d'échec, Jenkins affiche un message d'erreur.



```

Dashboard > Projet_DevOps_L3 > #7
[Status] [Changes] [Console Output] [Download] [Copy] [View as plain text]
[Edit Build Information] [Timings] [Git Build Data] [Pipeline Overview] [Pipeline Console] [Thread Dump] [Pause/resume] [Replay] [Pipeline Steps] [Workspaces] [Previous Build]

Started by user Afdel Desmond KOMBOU
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Projet_DevOps_L3
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Cloner le dépôt)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Projet_DevOps_L3/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Afdell1/projet_DevOps_L3.git # timeout=10
Fetching upstream changes from https://github.com/Afdell1/projet_DevOps_L3.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/Afdell1/projet_DevOps_L3.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision 1504fe9321b973d59abb2ceb5d7283d90febf5a3 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 1504fe9321b973d59abb2ceb5d7283d90febf5a3 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D master # timeout=10
> git checkout -b master 1504fe9321b973d59abb2ceb5d7283d90febf5a3 # timeout=10
Commit message: "Update docker-compose.yml"
> git rev-list --no-walk 1504fe9321b973d59abb2ceb5d7283d90febf5a3 # timeout=10

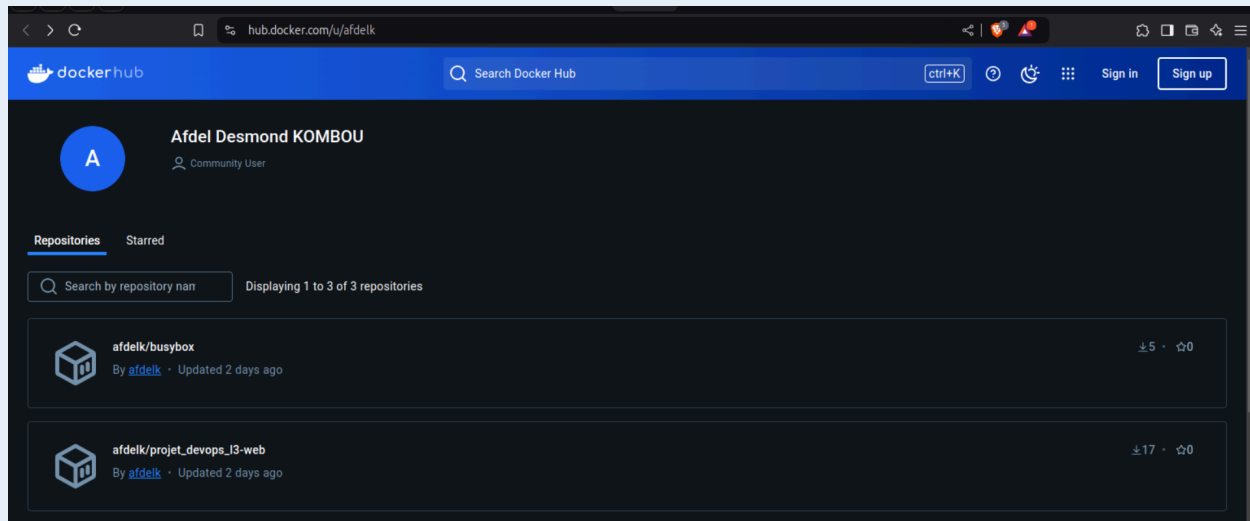
```

❖ Construire les conteneurs à l'aide de Docker.

Dashboard > Projet_DevOps_L3 > #7 > Pipeline Steps			
Status	Step	Arguments	Status
	Start of Pipeline - (34 sec in block)		✓
	node - (33 sec in block)		✓
	node block - (33 sec in block)		✓
	stage - (1.1 sec in block)	Cloner le dépôt	✓
	stage block (Cloner le dépôt) - (1 sec in block)		✓
	git - (0.92 sec in self)		✓
	stage - (19 sec in block)	Construire les conteneurs	✓
	stage block (Construire les conteneurs) - (19 sec in block)		✓
	sh - (19 sec in self)	docker-compose build	✓
	stage - (12 sec in block)	Démarrer les conteneurs	✓
	stage block (Démarrer les conteneurs) - (12 sec in block)		✓
	sh - (12 sec in self)	docker-compose up -d	✓
	stage - (0.21 sec in block)	Declarative: Post Actions	✓
	stage block (Declarative: Post Actions) - (0.16 sec in block)		✓
	echo - (59 ms in self)	Pipeline terminé.	✓
	echo - (50 ms in self)	Pipeline réussi !	✓

❖ Déployer l'application localement à l'aide de docker-compose.

stage - (19 sec in block)	Construire les conteneurs	✓
stage block (Construire les conteneurs) - (19 sec in block)		✓
sh - (19 sec in self)	docker-compose build	✓
stage - (12 sec in block)	Démarrer les conteneurs	✓
stage block (Démarrer les conteneurs) - (12 sec in block)		✓
sh - (12 sec in self)	docker-compose up -d	✓
stage - (0.21 sec in block)	Declarative: Post Actions	✓
stage block (Declarative: Post Actions) - (0.16 sec in block)		✓
echo - (59 ms in self)	Pipeline terminé.	✓
echo - (50 ms in self)	Pipeline réussi !	✓



```
afdel@afdel:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS			NAMES	
465de501961e	projet_devops_l3-web	"python app.py"	4 minutes ago	Up 4 m
inutes	0.0.0.0:5000->5000/tcp, :::5000->5000/tcp		projet_devops_l3-web-1	
450266efb106	busybox	"sleep infinity"	25 minutes ago	Up 25
minutes			projet_devops_l3-db-1	

```
afdel@afdel:~$
```

V. Déploiement avec Kubernetes

1. Créez un fichier deployment.yaml

Ce fichier deployment.yaml est utilisé pour déployer une application web et une base de données SQLite dans un cluster Kubernetes. Voici une explication simplifiée des sections principales :

a. Déploiement de l'application web :

Nom : web-app

Réplicas : 3 instances de l'application seront déployées.

Image : Utilise l'image Docker afdelk/projet_devops_l3-web:latest.

Port : L'application écoute sur le port 5000.

Variables d'environnement : Configure l'emplacement de la base de données SQLite.

Volumes : Utilise un volume temporaire (emptyDir) pour stocker les données de la base de données SQLite.

b. Service pour l'application web :

Nom : web-app-service

Type : ClusterIP (accessible uniquement à l'intérieur du cluster).

Port : Expose le port 5000 de l'application.

Déploiement de la base de données SQLite :

Nom : db-deployment

Réplicas : 1 instance de la base de données.

Image : Utilise l'image busybox pour exécuter un conteneur en mode veille.

Volumes : Utilise un volume temporaire pour stocker les données de la base de données.

c. Service pour la base de données :

Nom : db-service

Port : Expose le port 3306 pour la base de données.

Le fichier permet de déployer et de gérer l'application web et la base de données dans un environnement Kubernetes, en assurant la scalabilité et la connectivité entre les services.

```

Déploiement de l'application web
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web-app
          image: afdelk/projet_devops_l3-web:latest
          ports:
            - containerPort: 5000
          env:
            - name: sqlite-db
              value: "sqlite:///app/data/site.db"
          volumeMounts:
            - name: sqlite-data
              mountPath: /app/data
      volumes:
        - name: sqlite-data
          emptyDir: {}

```

2. Déployer l'application et la base de données sur un cluster Kubernetes.

```

afdel@afdel:~/Desktop/projet_DevOps_L3/projet_DevOps_L3$ nano deployment.yaml
afdel@afdel:~/Desktop/projet_DevOps_L3/projet_DevOps_L3$ minikube start
🌻 minikube v1.34.0 on Ubuntu 24.04
👉 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
👉 Pulling base image v0.0.45 ...
👉 minikube 1.35.0 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.35.0
💡 To disable this notice, run: 'minikube config set WantUpdateNotification false'

🔄 Restarting existing docker container for "minikube" ...
🐳 Preparing Kubernetes v1.31.0 on Docker 27.2.0 ...
🔍 Verifying Kubernetes components...
   ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
   ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🌟 Enabled addons: default-storageclass, storage-provisioner, dashboard
👉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
afdel@afdel:~/Desktop/projet_DevOps_L3/projet_DevOps_L3$ kubectl apply -f deployment.yaml
deployment.apps/web-app created
service/web-app-service created
deployment.apps/sqlite-db created
service/sqlite-db-service created

```

3. Configurer trois répliques pour l'application web.

```

GNU nano 7.2
# Déploiement de l'application web
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app

```

4. S'Assurer que la communication entre les conteneurs est fonctionnelle.

```

afdel@afdel:~/Desktop/projet_DevOPs_L3/projet_DevOPs_L3$ kubectl get services

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
db-service	ClusterIP	10.105.28.225	<none>	3306/TCP	9m4s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	59d
nginx-apache-service	NodePort	10.104.205.254	<none>	80:31496/TCP,8080:32193/TCP	56d
sqlite-db-service	ClusterIP	10.103.115.29	<none>	8080/TCP	33m
web-app-service	ClusterIP	10.107.220.38	<none>	80/TCP	9m4s
web-service	LoadBalancer	10.110.62.146	<pending>	5000:31825/TCP	2d23h

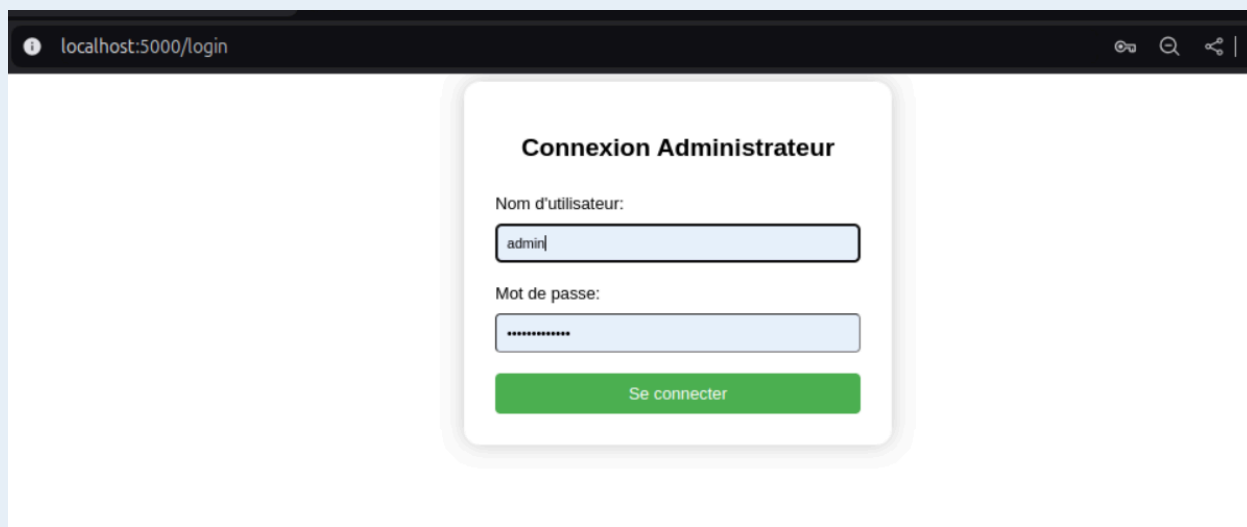
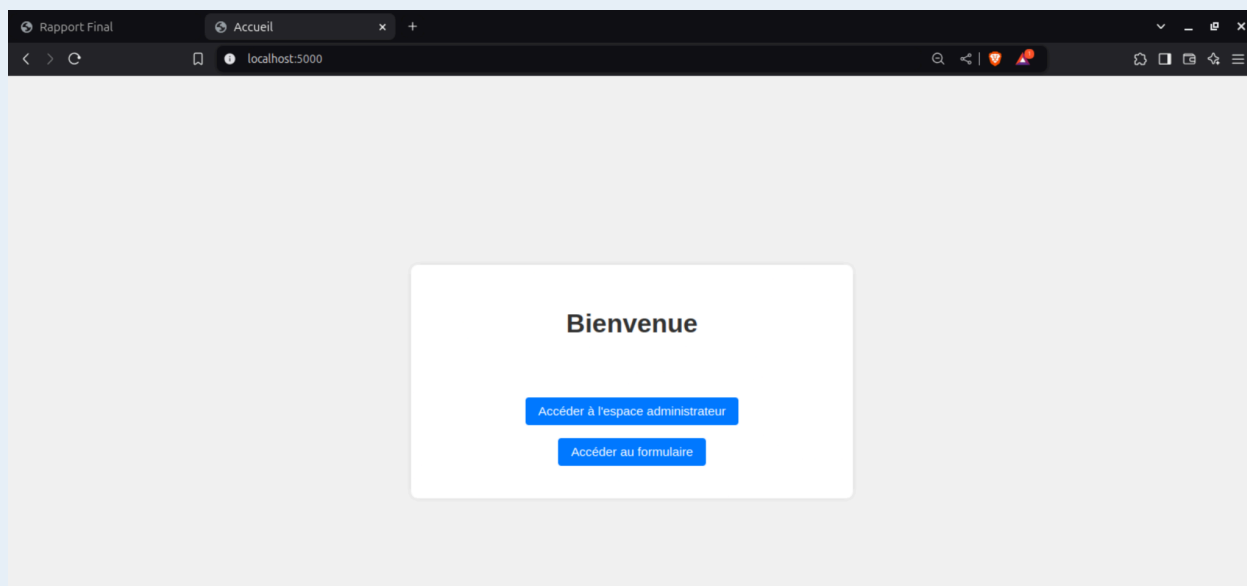
5. lancement de l'application après déploiement avec kubernetes

```

afdel@afdel:~/Desktop/projet_DevOPs_L3/projet_DevOPs_L3$ kubectl apply -f deployment.yaml
deployment.apps/web-app created
service/web-app-service created
deployment.apps/db-deployment created
service/db-service created
afdel@afdel:~/Desktop/projet_DevOPs_L3/projet_DevOPs_L3$ kubectl port-forward service/web-app-service 5000:5000
Forwarding from 127.0.0.1:5000 -> 5000
Forwarding from [::1]:5000 -> 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000
Handling connection for 5000

```

Résultats en localhost:5000

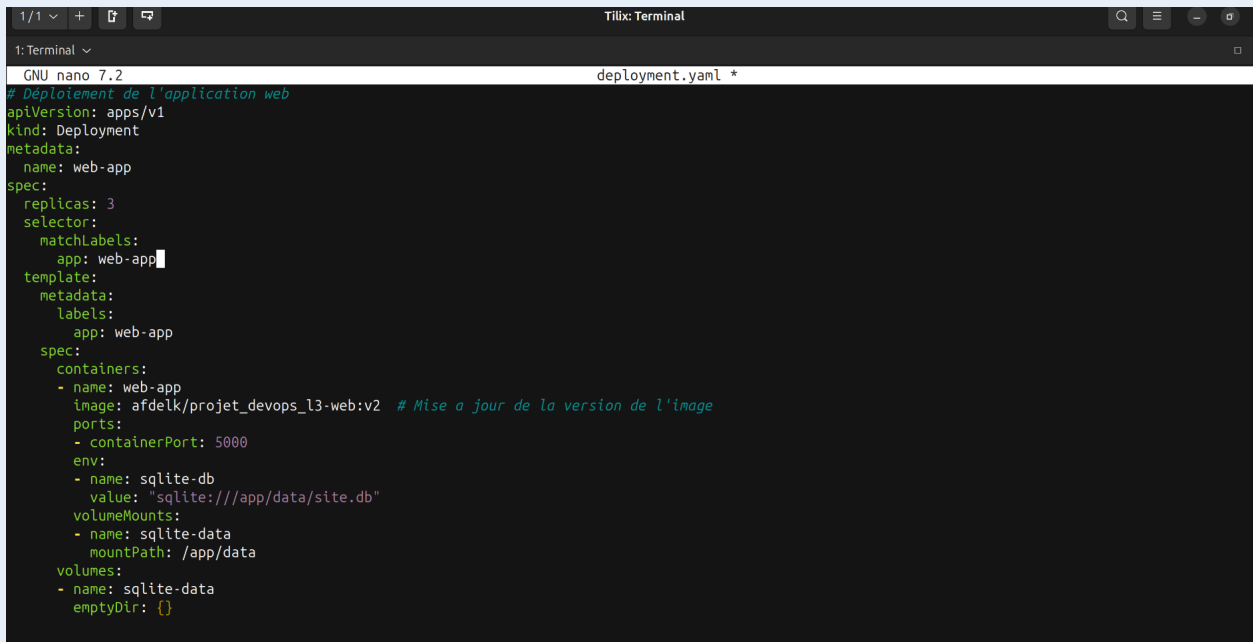


Mise à jour et Rendu Final de l'application

Pour modifier le fichier `deployment.yaml` afin de déployer la version 2 de l'application, vous devez mettre à jour l'image Docker utilisée dans le déploiement de l'application web. Voici les modifications nécessaires :

Changer l'image Docker :

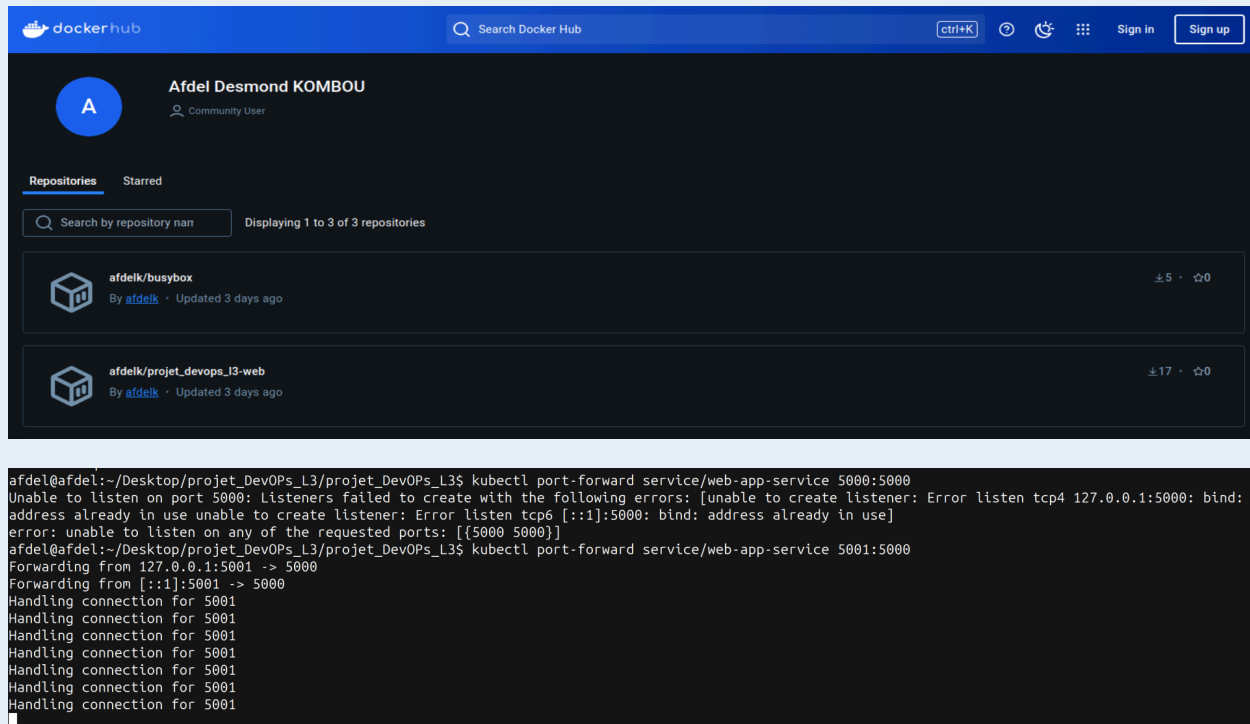
Nous avons remplacé l'image `afdelk/projet_devops_l3-web:latest` par la nouvelle image de la version 2, par `afdelk/projet_devops_l3-web:v2`.



```
1/1 ~ + deployment.yaml *
GNU nano 7.2
# Déploiement de l'application web
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web-app
          image: afdelk/projet_devops_l3-web:v2 # Mise à jour de la version de l'image
          ports:
            - containerPort: 5000
          env:
            - name: sqlite-db
              value: "sqlite:///app/data/site.db"
          volumeMounts:
            - name: sqlite-data
              mountPath: /app/data
      volumes:
        - name: sqlite-data
          emptyDir: {}
```

Mettre à jour le déploiement de l'application web :

Modifiez la section du déploiement de l'application web pour utiliser la nouvelle image.



Design et Mise en Page :

- Utilisation de **Tailwind CSS** pour une interface moderne et responsive.
- Amélioration de l'espacement, des bordures et des ombres pour une meilleure hiérarchie visuelle.
- Boutons radio stylisés en onglets cliquables avec des effets de hover et de sélection.

Expérience Utilisateur :

- Transitions fluides et animations subtiles pour une interaction plus agréable.
- Focus states visibles pour améliorer l'accessibilité.
- Disposition flexible des éléments pour s'adapter à toutes les tailles d'écran.

Conservation de la Logique :

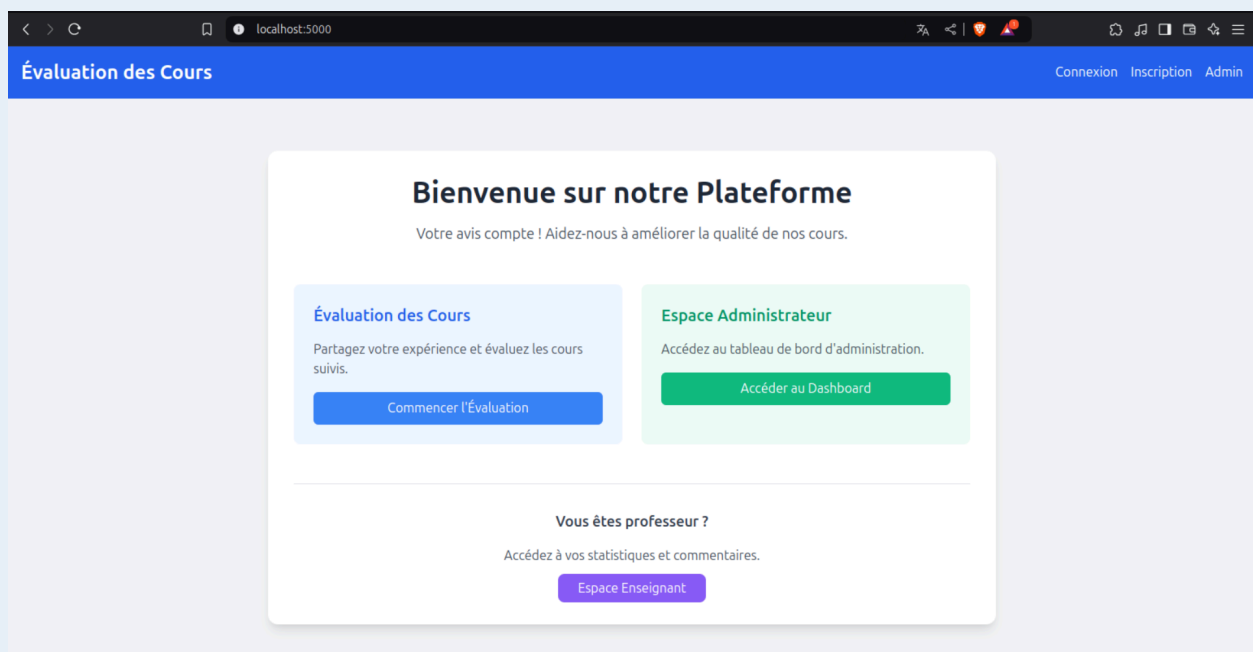
- Structure HTML et noms de champs inchangés pour assurer la compatibilité avec le backend.
- Script JavaScript conservé tel quel, avec seulement une mise à jour des classes CSS.

Responsive Design:

- Adaptation à toutes les résolutions d'écran, des mobiles aux grands écrans.
- Disposition des boutons de choix optimisée pour les petits écrans.

Les modifications se concentrent sur l'amélioration visuelle et l'expérience utilisateur, sans altérer la logique ou la fonctionnalité existante de l'application.

1. Nouvelle Page d'accueil



2. Nouvelle Page de sélection de la matière et de la salle de classe

localhost:5000/class_subject

Choisissez la Classe et la Matière

Classe :

Licence 1

Matière :

Anglais

Sélectionner une matière

Mathématiques

Anglais

SQL

3. Nouvelle Page d'evaluation

localhost:5000/survey

Évaluation du cours

Quel est votre niveau d'implication dans le cours ?

0 1 2 3 4 5 6 7 8 9 10

Quel était votre niveau de compétences/connaissances en début ?

0 1 2 3 4 5 6 7 8 9 10

Quel est votre niveau de compétences/connaissances actuel ?

0 1 2 3 4 5 6 7 8 9 10

Comment évaluez-vous la motivation et les échanges avec le professeur ?

0 1 2 3 4 5 6 7 8 9 10

Comment trouvez-vous les outils et méthodologies développés par le professeur ?

0 1 2 3 4 5 6 7 8 9 10

Veillez noter les exemples employés et les exercices réalisés

The screenshot shows a web browser window with the address bar displaying 'localhost:5000/survey'. The survey form contains the following elements:

- A header bar with navigation links: 'Accueil', 'Inscription', 'Administration', and 'Évaluer un Cours' (highlighted in green).
- A title 'Évaluation des Cours'.
- A question: 'Veuillez noter la pertinence et la clarté des explications' with a rating scale from 0 to 10. The number 10 is selected.
- A question: 'Pensez-vous être capable de mettre en pratique les acquisitions ?' with a dropdown menu showing 'Oui'.
- A question: 'Êtes-vous satisfait(e) des conditions d'organisation du cours (salle, installation, matériels, etc.) ?' with a dropdown menu showing 'Oui'.
- A question: 'Quel est votre niveau de satisfaction générale ?' with a rating scale from 0 to 10.
- A text area for 'Commentaires supplémentaires' containing the text: 'Thanks for this lesson, i'm proud to attend a your class, it was a great lessons|'.
- A blue button labeled 'Soumettre'.

4. Nouvelle Page de Connexion à l'espace administrateur

The screenshot shows a web browser window with the address bar displaying 'localhost:5000/login'. The page features a blue header bar with the title 'Évaluation des Cours' and navigation links: 'Accueil', 'Inscription', 'Administration', and 'Évaluer un Cours' (highlighted in green).

The main content area displays a login form titled 'Connexion Administrateur' with the following fields and elements:

- A label 'Nom d'utilisateur' above a text input field containing 'admin|'.
- A label 'Mot de passe' above a password input field showing masked characters '*****'.
- A blue button labeled 'Se connecter'.

5. Nouvelle Page d'Administration

Administration

Accueil Modifier les identifiants Afficher les réponses du sondage Se déconnecter

Générer un Rapport

Sélectionnez une classe : Licence 1

Sélectionnez une matière : Licence 1

Générer le Rapport

Gestion des Matières et Classes

Ajouter une Matière

Nom de la matière :

Statistique

Ajouter

Supprimer une Matière

Sélectionnez une matière :

Mathématiques

Supprimer

Ajouter une Classe

Nom de la classe :

licence 3 BD

Ajouter

Supprimer une Classe

Sélectionnez une classe :

localhost:5000/report

Motivation et échanges avec le professeur	7.0
Outils et méthodologies développés	8.0
Exemples employés et exercices réalisés	9.0
Pertinence et clarté des explications	10.0

3. Mesure de la capacité de mise en pratique des compétences acquises

Critère	Pourcentage
Pourcentage des étudiants capables de mettre en pratique les compétences	100.0%

4. Aspects du cours et Satisfaction générale

Critère	Note Moyenne / 10
Satisfaction avec l'organisation du cours	10.0
Satisfaction générale moyenne	9.0

6. Nouvelle Page de réponse aux sondages

Réponses au Sondage [Retour](#)

Filtrer par classe : Toutes les classes [Filtrer](#)

Classe	Matière	Commentaire
Licence 1	Mathématiques	merci pour ces leçons, je suis heureux d'assister a vos cours
Licence 2	Anglais	
Licence 3	SQL	erci pour le cours
Master 2	Mathématiques	le prof est excellent
Master 1	Anglais	trop content

[Réinitialiser les Résultats](#)

7. Nouvelle Page de changement ds identifiants Administrateurs

Changer les Identifiants Administrateurs

Ancien Nom d'utilisateur :

Ancien Mot de Passe :

Nouveau Nom d'utilisateur :

Nouveau Mot de Passe :

[Retour à l'administration](#) [Mettre à jour](#)

CONCLUSION

Le développement de ce **système d'évaluation des enseignements** pour le Dakar Institute of Technology (DIT) marque une avancée significative dans l'optimisation des processus académiques. En offrant une solution **sécurisée**, **flexible** et **automatisée**, nous avons permis une meilleure gestion des retours pédagogiques, un contrôle total des données et une analyse approfondie des évaluations.

D'un point de vue technique, ce projet nous a permis d'appliquer des concepts clés du DevOps :

- **Développement web avec Flask,**
- **Automatisation CI/CD avec Jenkins,**
- **Conteneurisation avec Docker,**
- **Orchestration et scalabilité avec Kubernetes.**

Au-delà des aspects techniques, cette expérience nous a enseigné l'importance d'une gestion de projet efficace et d'une collaboration fluide au sein d'une équipe. Grâce à **Trello**, **Slack** et **GitHub**, nous avons pu structurer nos tâches et assurer un suivi rigoureux du travail.

Bien que le projet soit fonctionnel, des améliorations futures sont envisageables, notamment l'intégration d'une **intelligence artificielle** pour **analyser les tendances des évaluations** ou **encore le renforcement de la sécurité des accès**.

Nous remercions **M. Madické Diop** pour son encadrement ainsi que tous les acteurs du DIT qui ont contribué à cette initiative. Cette expérience a renforcé nos compétences et nous encourage à poursuivre notre exploration du monde du DevOps et du développement logiciel.

 **Un pas de plus vers une éducation modernisée et pilotée par la donnée !**

NOS REMERCIEMENTS 😊